

# Hibernate

## get() vs getReference()

### Before Starting

In the previous class, we already created the Hibernate configuration, Entity class, SessionFactory, Session and Transaction.

For every Hibernate operation, we need a connection/session setup.

Instead of writing the same SessionFactory creation code again and again, we created a separate Singleton class:

ConnectionEstablisher

```
package com.nit.connectionProvider;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class ConnectionEstablisher {

    private static SessionFactory factory=null;
    private ConnectionEstablisher() {

    }

    public static SessionFactory getConnection() {
        if(factory==null) {
            Configuration configuration=new Configuration();
            configuration.configure
            ("resources/hibernate-cnf.xml");

            factory= configuration.buildSessionFactory();
            System.out.println(factory);
        }
        return factory;
    }
}
```

The main purpose of this class is:

```
-> Create SessionFactory only once
-> Reuse the same SessionFactory for multiple operations
-> Avoid writing the same connection/configuration code again and again
```

So in our programs, we simply use:

```
ConnectionEstablisher.getConnection().openSession();
```

## OUR ENTITY

We are using the Student entity:

```
@Entity
@Table(name="student_values")
@Data
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private int id;

    @Column(name="student_name")
    private String name;

    @Column(name="student_degree")
    private String degree;
}
```

The annotations are already covered in the previous class.

Our table contains student information like:

| ID | NAME  | DEGREE |
|----|-------|--------|
| 1  | abc   | B.tech |
| 2  | xyz   | MCA    |
| 3  | Virat | B.tech |

## GET()

### Program

```
Session session =
    ConnectionEstablisher
        .getConnection()
        .openSession();

Transaction transaction =
    session.beginTransaction();

System.out.println(
```

```

    "Fetching the data from Old version method"
);

Student student =
    session.get(Student.class, 3);

System.out.println(
    "Id of the student " + student.getId()
);

System.out.println(
    "Name of the student " + student.getName()
);

System.out.println(
    "Qualification : " + student.getDegree()
);

```

## What happens when we call get()?

When we write:

```

Student student =
    session.get(Student.class, 3);

```

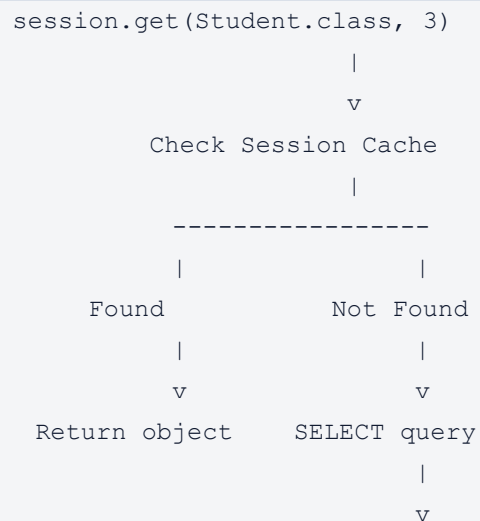
we are basically telling Hibernate:

“Hibernate, give me the Student object whose ID is 3.”

Hibernate directly checks whether the entity is available in the current Session.

If it is not already available, Hibernate goes to the database and executes a SELECT query.

Conceptually:



```
Student object
|
v
Return it
```

## Example

```
Student student =
    session.get(Student.class, 3);
```

If ID 3 exists:

```
SELECT ...
FROM student_values
WHERE id = 3;
```

Then Hibernate gives us the Student object.

After that we can access:

```
student.getId();
student.getName();
student.getDegree();
```

## What if the student does not exist?

Suppose we write:

```
Student student =
    session.get(Student.class, 100);
```

and there is no student with ID 100.

Then:

```
get()
|
v
Student not found
|
v
null
```

So:

```
get() -> returns null if the entity does not exist.
```

## IMPORTANT POINT ABOUT CACHE

One important thing to understand is that Hibernate has a First-Level Cache associated with the Session.

If we call:

```
Student student1 =  
    session.get(Student.class, 3);
```

and again:

```
Student student2 =  
    session.get(Student.class, 3);
```

within the same Session, Hibernate can get the entity from the First-Level Cache instead of hitting the database again.

```
get()  
-> checks the current Session cache first.  
-> if the entity is not available there, Hibernate executes  
    a SELECT query and loads the entity.
```

## getReference()

In newer Hibernate/JPA versions, instead of the old:

```
session.load()
```

we can use:

```
session.getReference()
```

Our program is:

```
Session currentSession =  
    ConnectionEstablisher  
        .getConnection()  
        .openSession();  
  
System.out.println(  
    "Fetching the data from updated one"  
);  
  
Student student =  
    currentSession.getReference(Student.class, 3);  
  
System.out.println(  
    "Id of the student " + student.getId()  
);  
  
System.out.println(  
    "Id of the student " + student.getId()  
);
```

```

        "Name of the student " + student.getName()
    );

    System.out.println(
        "Qualification : " + student.getDegree()
    );

```

## What is getReference()?

When we write:

```

Student student =
    session.getReference(Student.class, 3);

```

we are basically telling Hibernate:

"I need a reference to the Student whose ID is 3."

Hibernate can return a proxy/reference instead of immediately loading the complete Student data.

Conceptually:

```

getReference()
    |
    v
Proxy / Reference
    |
    v
ID is already known
    |
    v
Actual data can be loaded
when it is required

```

## IMPORTANT: getReference() AND SELECT QUERY

```

getReference()
    -> returns a proxy/reference when possible.
    -> Hibernate loads the actual entity data when that
        data is required.
    -> the ID may already be available in the proxy.

```

## getId() WITH getReference()

For example:

```
Student student =
    session.getReference(Student.class, 3);

System.out.println(student.getId());
```

The proxy already knows:

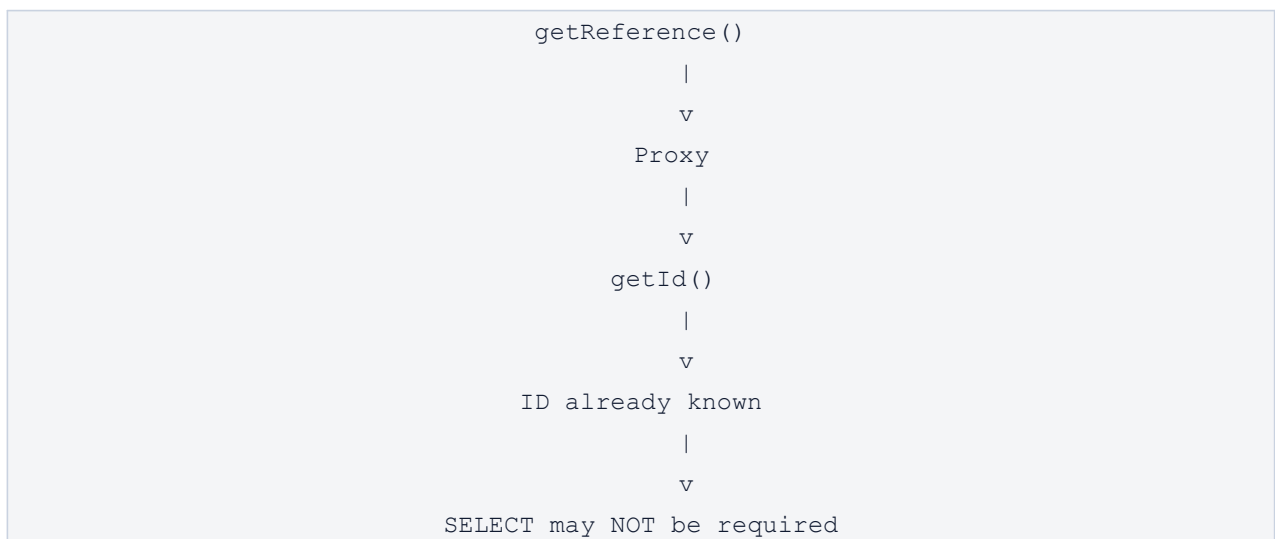
```
ID = 3
```

because we requested:

```
getReference(Student.class, 3)
```

Therefore Hibernate may not need to execute a SELECT just to give us the ID.

So:



This is why, for a classroom demonstration, we should not use only:

```
student.getId();
```

to demonstrate lazy loading/proxy initialization.

## getName() WITH getReference()

Now consider:

```
Student student =
    session.getReference(Student.class, 3);

System.out.println(
    student.getName()
);
```

The proxy already knows the ID:

But it does not necessarily have:

```
name
degree
```

So Hibernate may need to initialize the proxy.

Then it can execute:

```
SELECT ...
```

and load the actual Student data.

Conceptually:

```
getReference()
  |
  v
Proxy
  |
  v
getName()
  |
  v
Hibernate needs
Student data
  |
  v
SELECT query
  |
  v
Actual Student data
  |
  v
getName()
```

So for practical demonstration, use:

```
student.getName();
```

or:

```
student.getDegree();
```

after `getReference()`.

### 3. `get()` returns null if entity does not exist

Suppose ID 100 does not exist:



```
Student student =  
    session.get(Student.class, 100);
```

Result:

```
student = null
```

So:

```
get()  
    -> entity exists      -> Student object  
    -> entity not exists -> null
```

#### 4. getReference() can throw an exception

Suppose ID 100 does not exist:

```
Student student =  
    session.getReference(Student.class, 100);
```

Initially, Hibernate may give us a proxy/reference.

The problem is discovered when Hibernate actually tries to initialize that proxy and load the entity.

For example:

```
System.out.println(student.getName());
```

At this point Hibernate may execute the SELECT and discover that the entity does not exist.

Then an exception can occur, commonly:

```
EntityNotFoundException
```

So remember:

```
get()  
    -> not found -> null  
  
getReference()  
    -> reference may be returned first  
    -> if actual entity is required and not found  
        -> exception can occur
```

### WHEN SHOULD WE USE get()?

Use get() when:

```
-> We actually need the entity data.  
-> We want to check whether the entity exists.  
-> We want null when the entity does not exist.  
-> We are going to use multiple properties of the entity.
```

-> We want straightforward and easy-to-understand code.

Example:

```
Student student =  
    session.get(Student.class, 3);  
  
if(student != null) {  
  
    System.out.println(student.getName());  
    System.out.println(student.getDegree());  
  
}
```

This is useful when we genuinely want to read the Student information.

## WHEN SHOULD WE USE `getReference()`?

Use `getReference()` when:

```
-> We mainly need a reference to an existing entity.  
-> We don't need the complete entity data immediately.  
-> We want to use the entity as a reference in another  
    operation.  
-> Loading the complete entity is unnecessary.
```